Matemática Discreta 2

Aula 10 - Algoritmos em Grafos (parte 1)

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Um grafo é

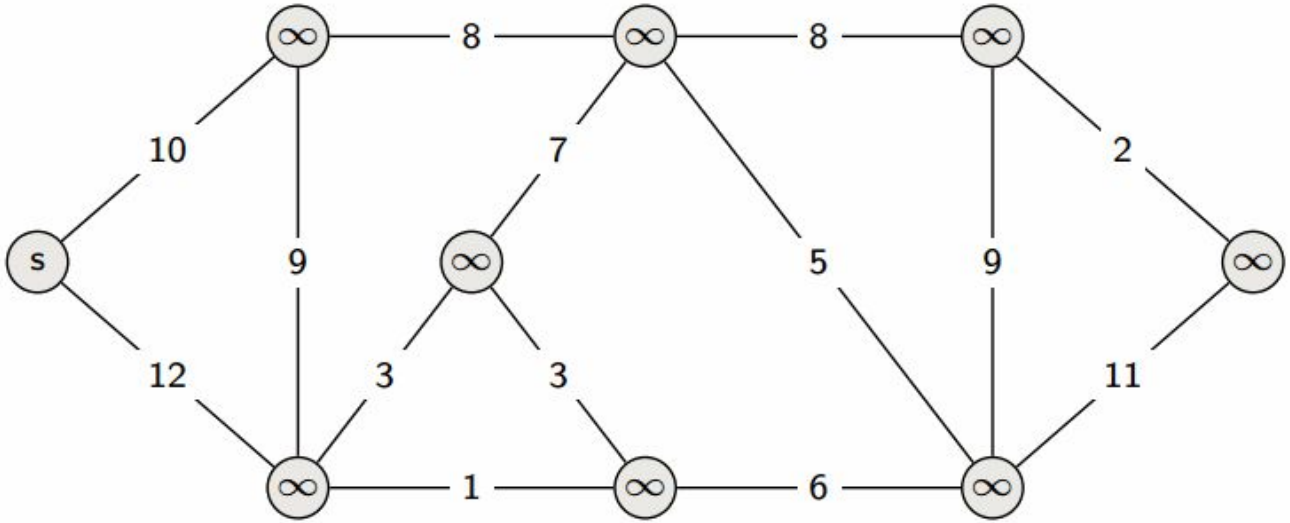
uma representação gráfica de

elementos de dados e

das conexões entre alguns destes itens

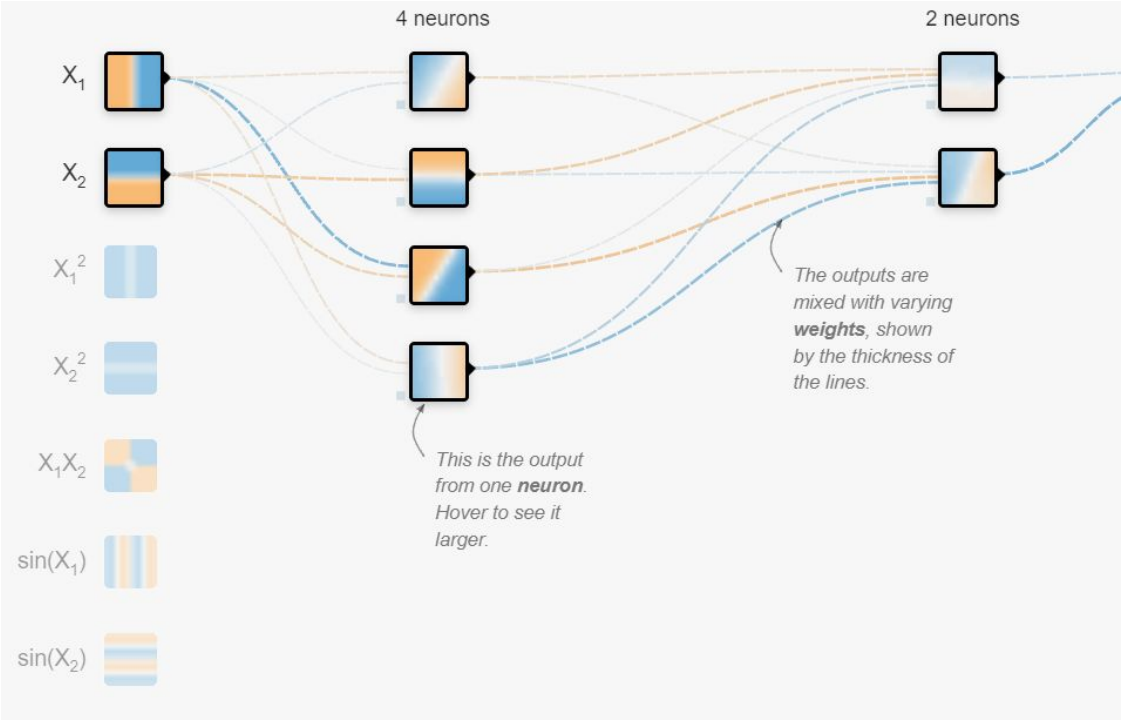
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Aplicações: Waze



An Introduction to Graph Theory. Before diving into graph theory, we… | by Michelle Venables | Towards Data Science

Aplicações: Redes Neurais

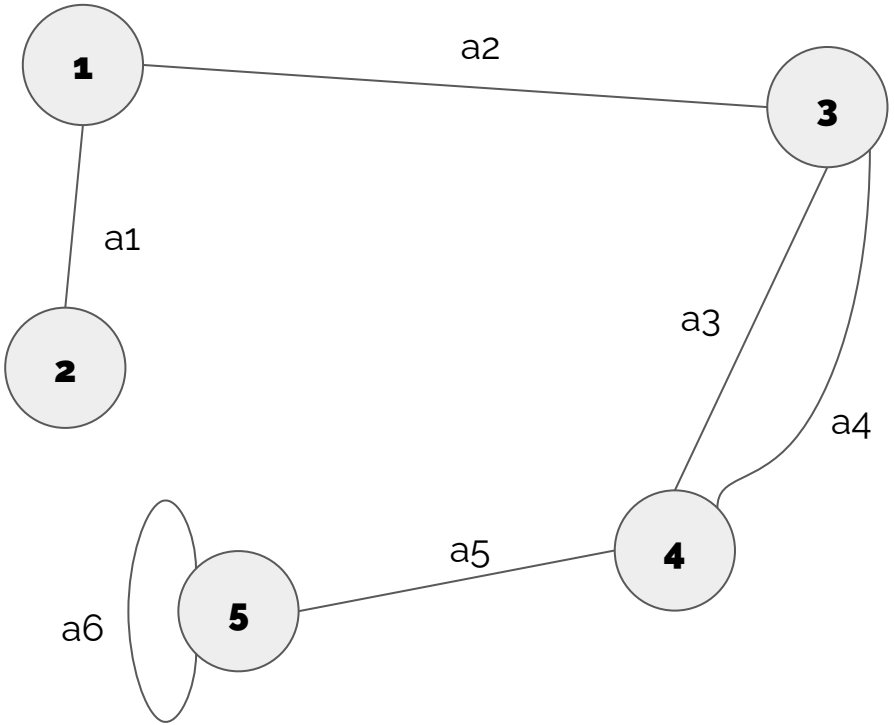


https://playground.tensorflow.org/

Trace um grafo

Que tenha os vértices {1, 2, 3, 4, 5}, as arestas {a1, a2, a3, a4, a5, a6} e a função

g(a1) = 1 -2,
g(a2) = 1-3,
g(a3) = 3-4,
g(a4) = 3-4,
g(a5) = 4-5 e
g(a6) = 5-5.

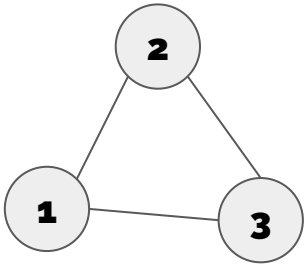


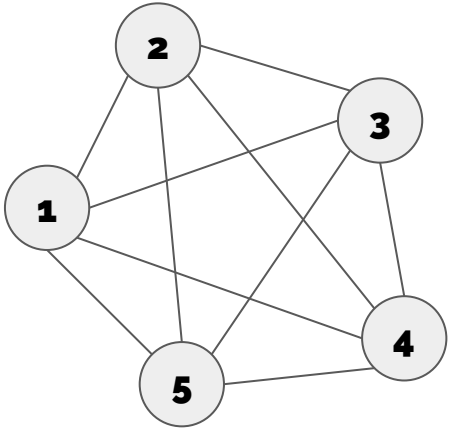
Grafo Completo

Um grafo completo é aquele no qual **todos os vértices distintos são adjacentes**

Geralmente é chamado de ***Kn***

Como podemos criar um grafo completo com 3 nós, e com 5?





Grafo Bipartido Completo

Grafos bipartidos completos são grafos em que os **vértices podem ser divididos em dois conjuntos disjuntos**, de tal forma que quaisquer dois vértices tomados no mesmo conjunto não são adjacentes, mas **todos dois vértices escolhidos em conjuntos diferentes são adjacentes**.

Geralmente chamados de ***Kn,m***

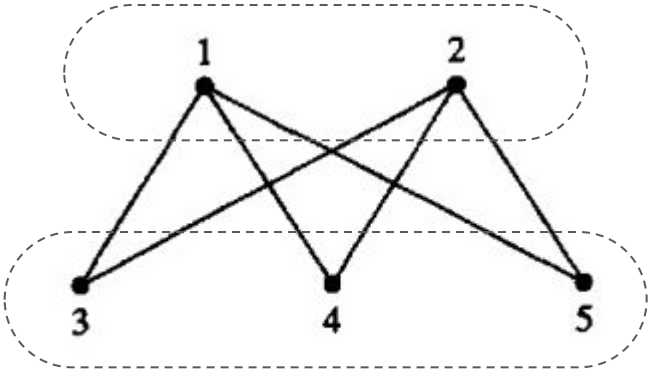
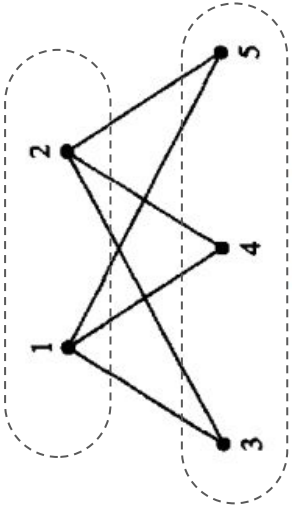


Figura 5.13



Árvores

Uma árvore é

um **grafo acíclico** e **conexo**

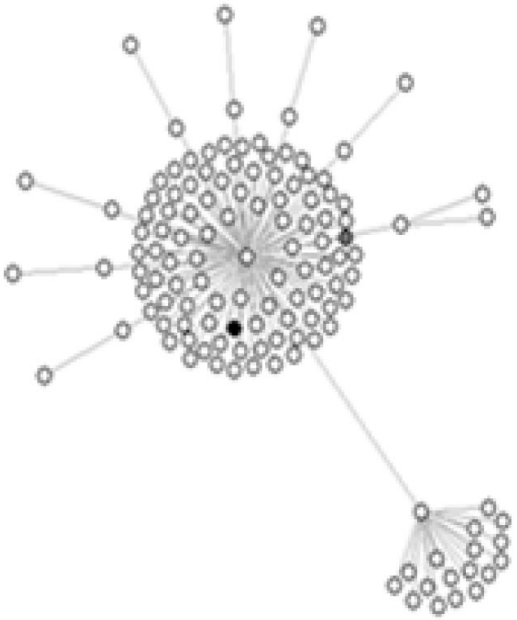
com um nó designado como a

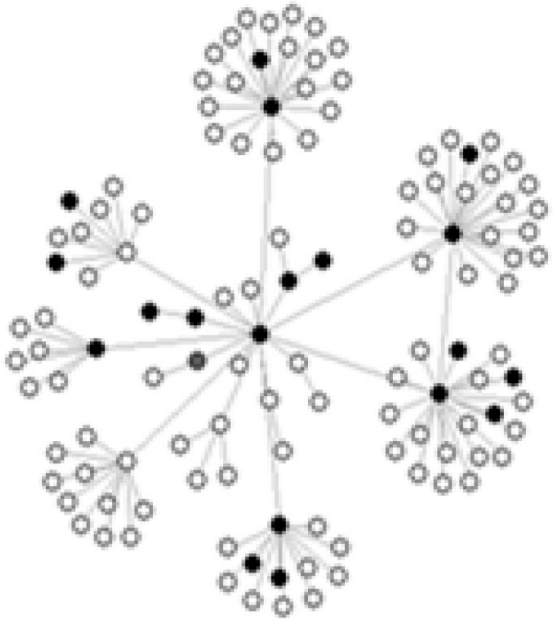
raiz da árvore

Como seria a representação visual de retweets?

Mima Diaz de Conduru 🚦🎀🔗 on Twitter: "Eu amo Bode Gaiato https://t.co/GdnXAbIAvn" / Twitter

Propagação de “Real vs Fake News” em retweets





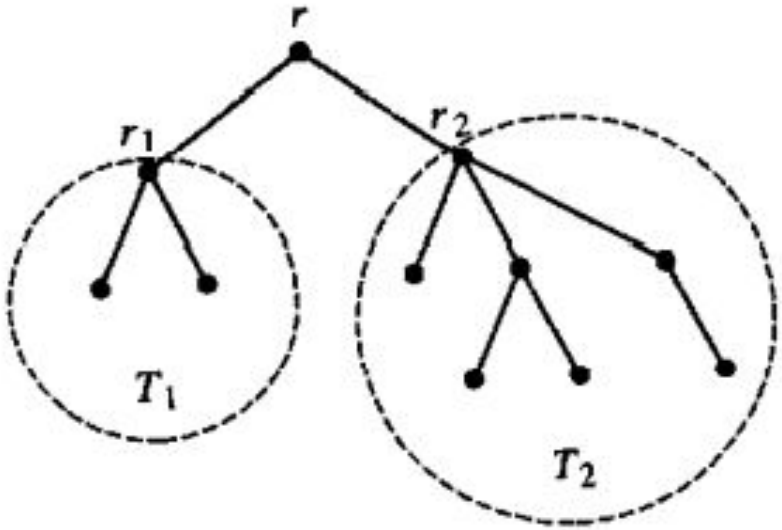
(a) TW propagation tree of fake news;

(b) TW propagation tree of real news

Electronics | Free Full-Text | Fake News Analysis Modeling Using Quote Retweet | HTML (mdpi.com)

Raiz, Vértices Pais e Filhos

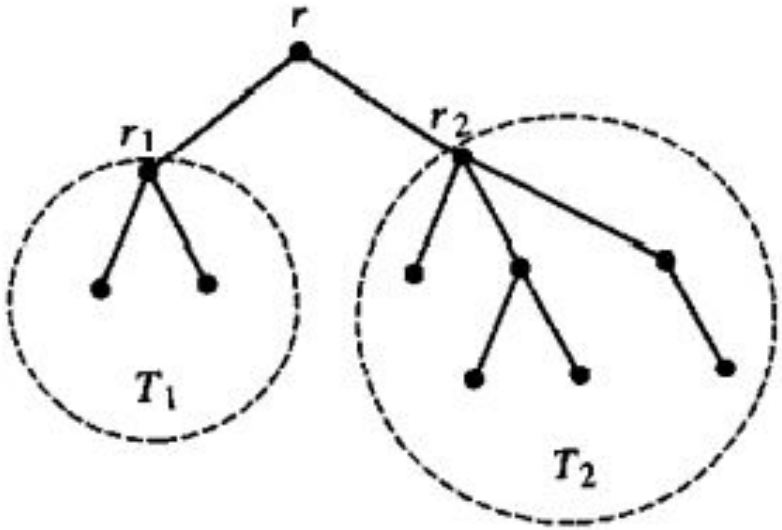
- **r** é a raiz da árvore ao lado
- os vértices **r1** e **r2** são filhos de **r**, e
- **r** é pai de **r1** e **r2**
- **r1** e seus filhos podem ser considerados uma sub-árvore **T1**
- o mesmo vale para **T2**
- os nós que não têm filhos são chamados de **folhas**



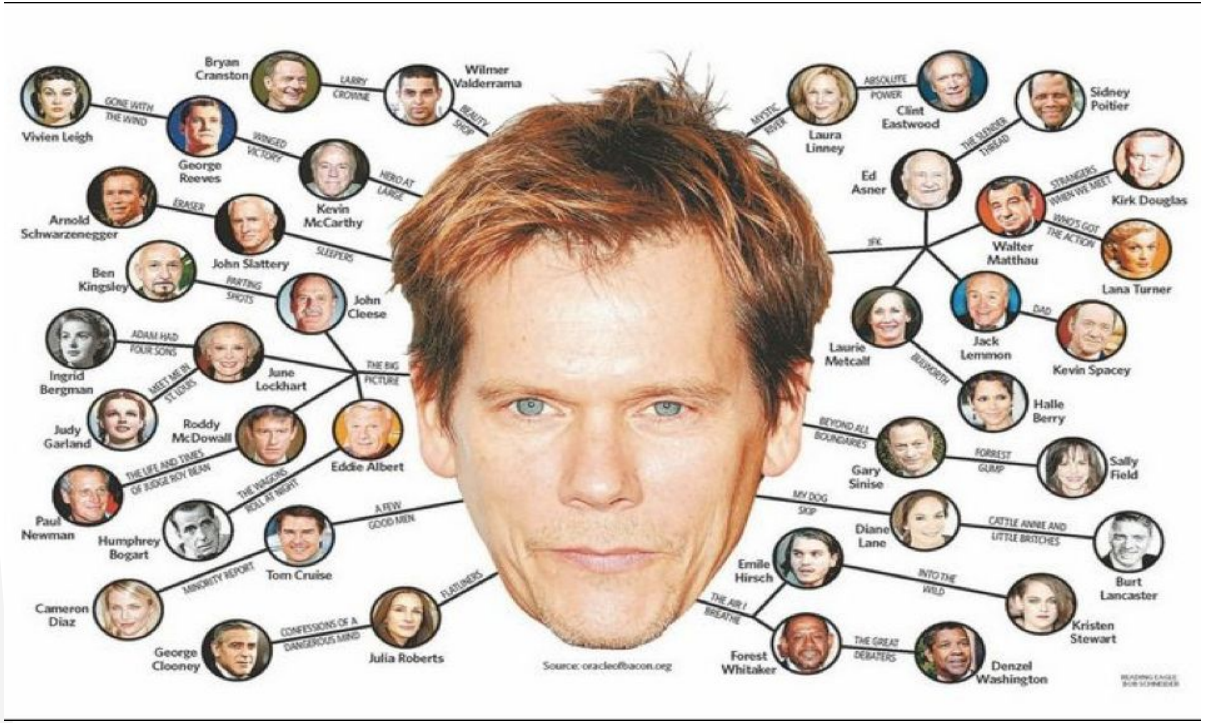
Profundidade de um Vértice

A profundidade de um vértice
em uma árvore é
o comprimento do caminho
da raiz até o vértice;

em particular,
a raiz tem profundidade **0**



Profundidade de um Vértice



Selton Mello has a Bacon number of 3.

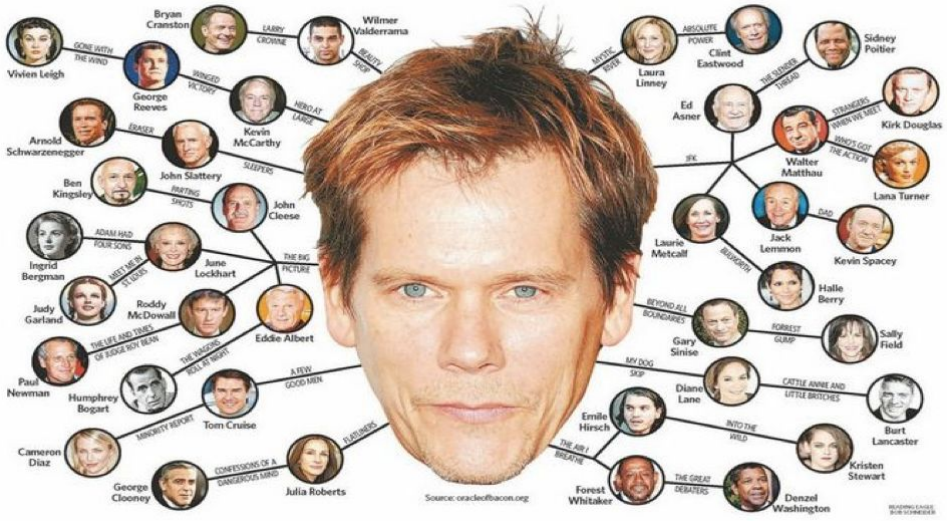
Selton Mello
was in
A Dog's Will
with
Matheus Nachtergaele
was in
Journey to the End of the Night
with
Brendan Fraser
was in
The Air I Breathe
with
Kevin Bacon

https://oracleofbacon.org/

Altura da Árvore

A altura (profundidade) da árvore é a maior profundidade de todos seus vértices; em outras palavras, é o comprimento do maior caminho entre a raiz e um vértice.

Qual é a altura/profundidade da árvore de Kevin Bacon?



Conceito de Hoje

Conceito de Hoje

Algoritmos em Grafos (parte 1)

Buscas em Árvores Binárias
Códigos de Huffman

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

buscas em bases de dados

Como buscar se um IG já está cadastrado?

@kevin

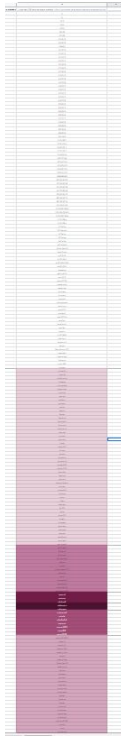
```
usuarios = {IG1, IG2, ..., IGn};
contaDisponivel = true;

for(Usuario usuario : usuarios) {
        if(usuario == @kevin) {
                print("IG não disponível");
                contaDisponilvel = false;
        }
}
```

|usuarios| = n, para descobrir que o login está disponível temos que realizar **n operações**

Como buscar se uma senha já está cadastrada?

NordPass | Features | Pricing | Apps | Business | Blog | Help | Login

Position	Password	Number of users	Time to crack it	Times exposed
1. ↑ (2)	123456	2,543,285	Less than a second	23,597,311
2. ↑ (3)	123456789	961,435	Less than a second	7,870,694
3. (new)	picture1	371,612	3 Hours	11,190
4. ↑ (5)	password	360,467	Less than a second	3,759,315
5. ↑ (6)	12345678	322,187	Less than a second	2,944,615
6. ↑ (17)	111111	230,507	Less than a second	3,124,368
7. ↑ (18)	123123	189,327	Less than a second	2,238,694
8. ↓ (1)	12345	188,268	Less than a second	2,389,787
9. ↑ (11)	1234567890	171,724	Less than a second	2,264,884
10. (new)	senha	167,728	10 Seconds	8,213

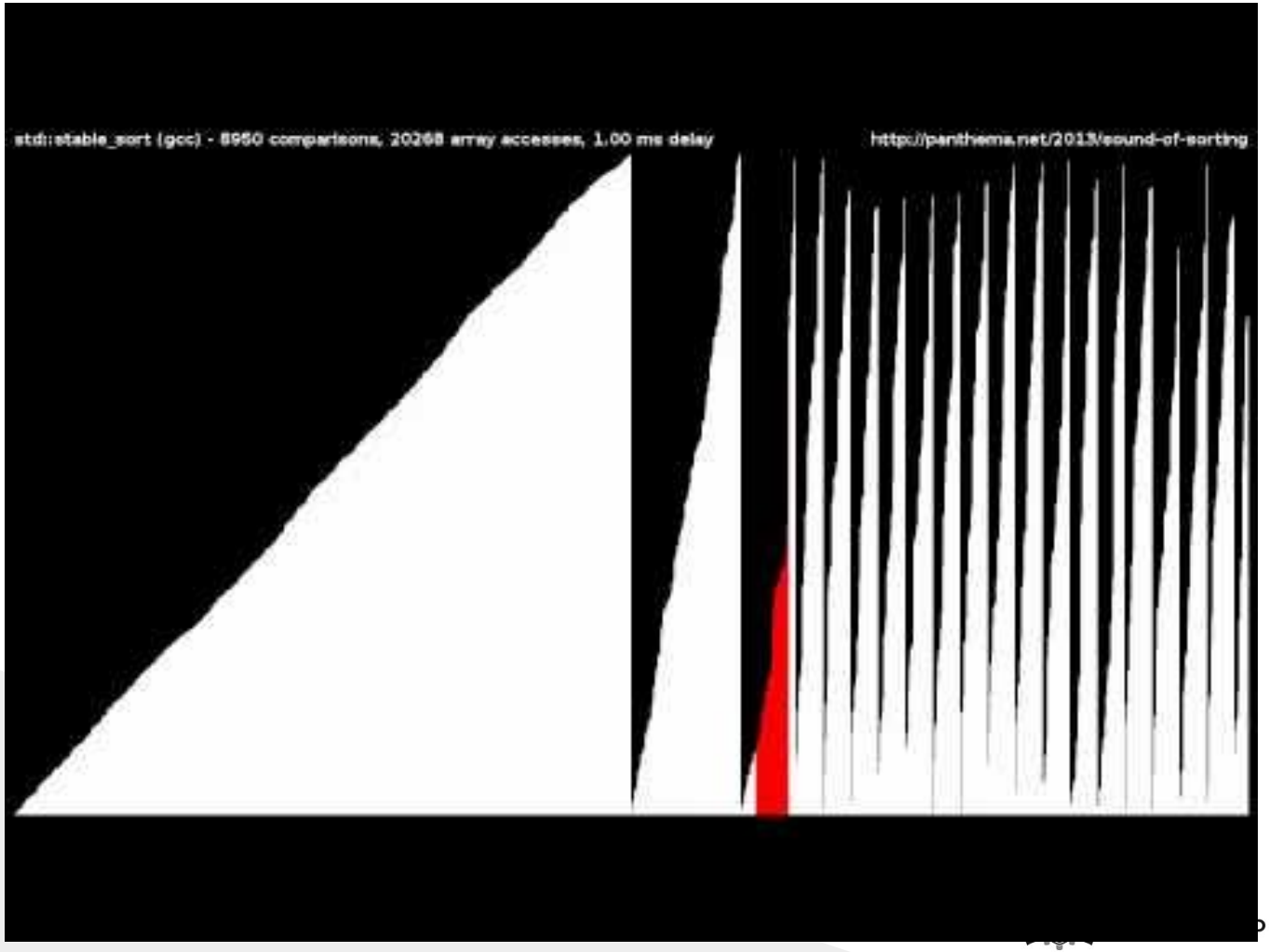


Lista das 200 senhas mais usadas:
https://nordpass.com/most-common-passwords-list

0
0
123
1111
1234
10203
12345
101010
102030
111111
112233
121212
123123
123321
123456
123456
123654
131313
142536
147258
159357
159753
222222
333333
456789

Top 200 Most Common Passwords of 2020 | NordPass

E se tivéssemos um conjunto de dados ordenado?



Busque o número 9 no array a seguir

| 2 | 5 | 8 | 9 | 10 | 12 | 14 |

árvores binárias

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

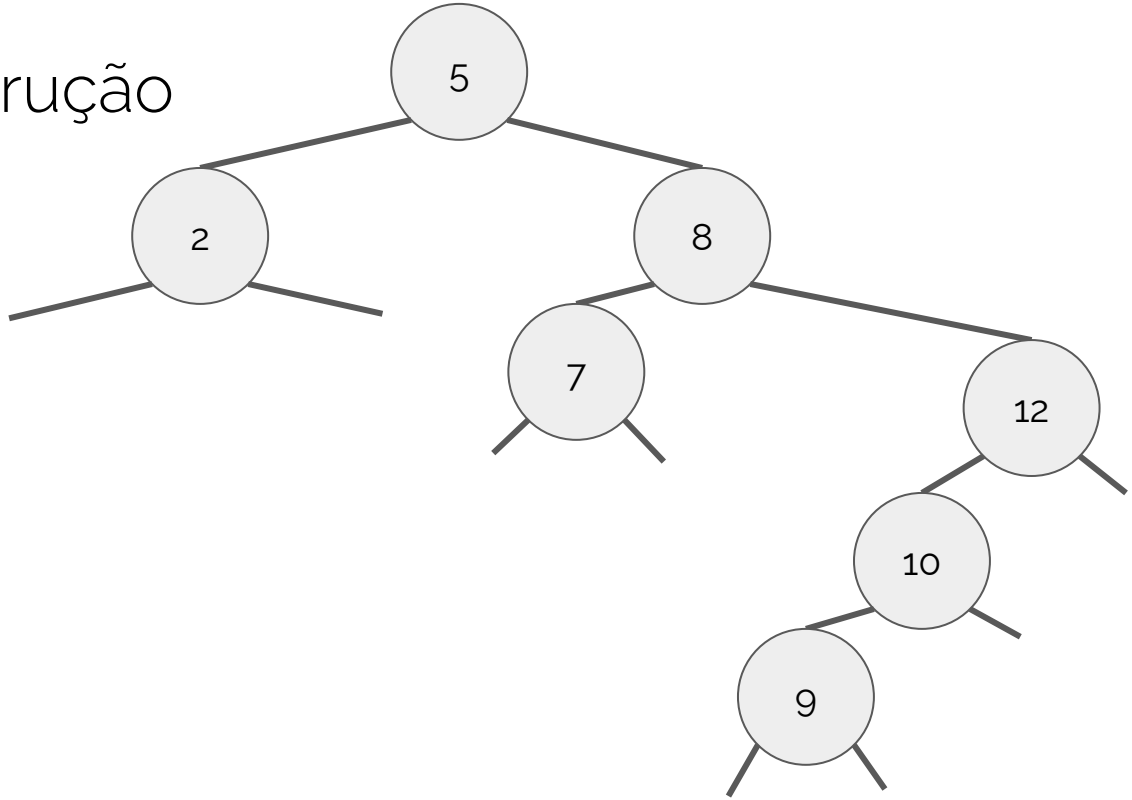
Árvores Binárias

São árvores em que cada nó tem **no máximo dois filhos**

Árvores Binárias: Construção

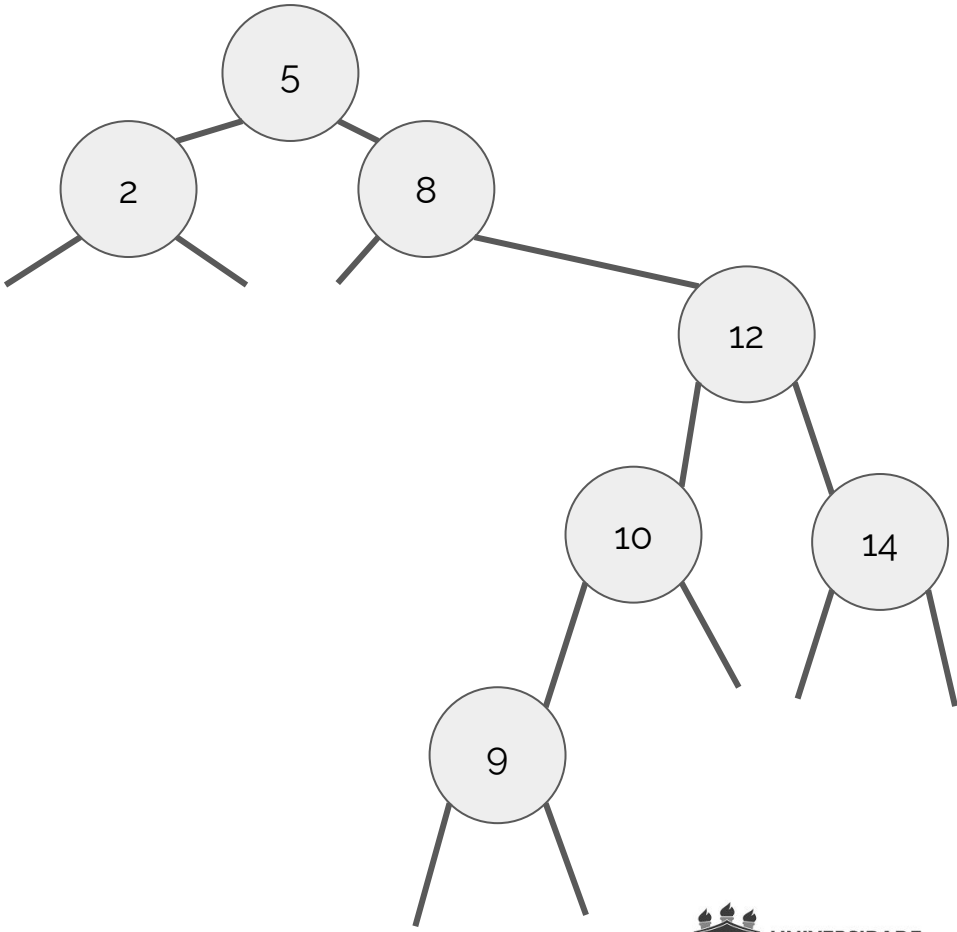
Crie uma árvore binária adicionando os elementos a seguir em ordem

5
8
2
12
10
14
9
7



Árvores Binárias: Prática

Busque o **número 14** na árvore criada



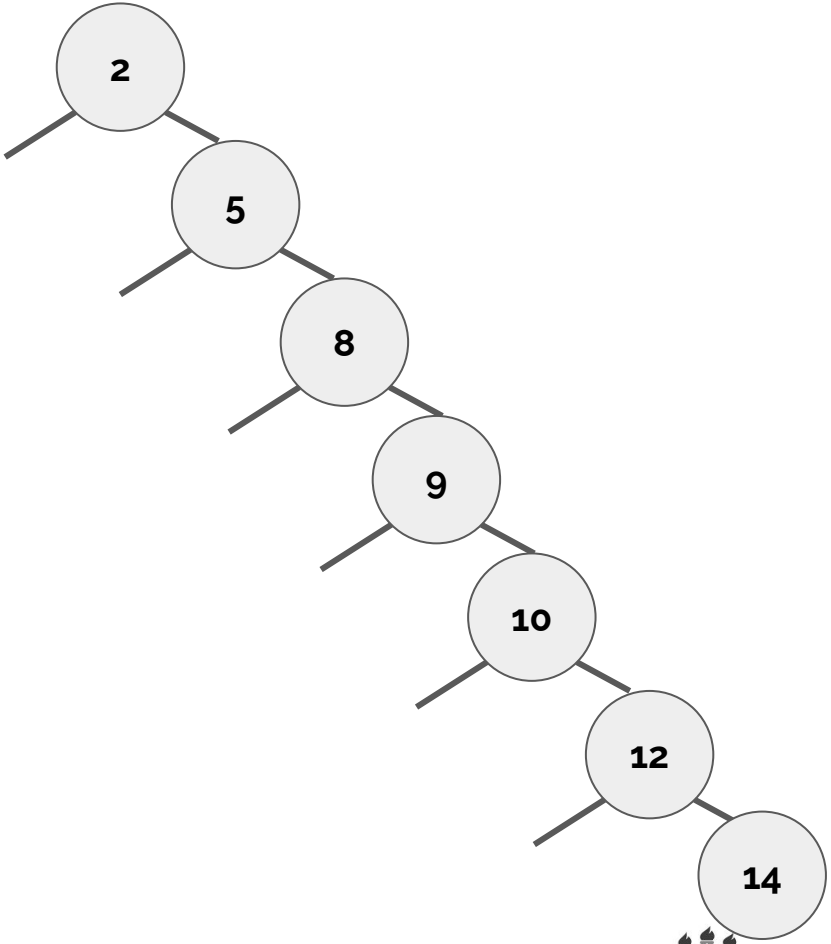
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Árvores Binárias: Construção

Crie uma árvore binária adicionando os elementos a seguir em ordem

2
5
8
9
10
12
14

O que aconteceu com a árvore? A busca do número 14 ainda é mais rápida nesta árvore?



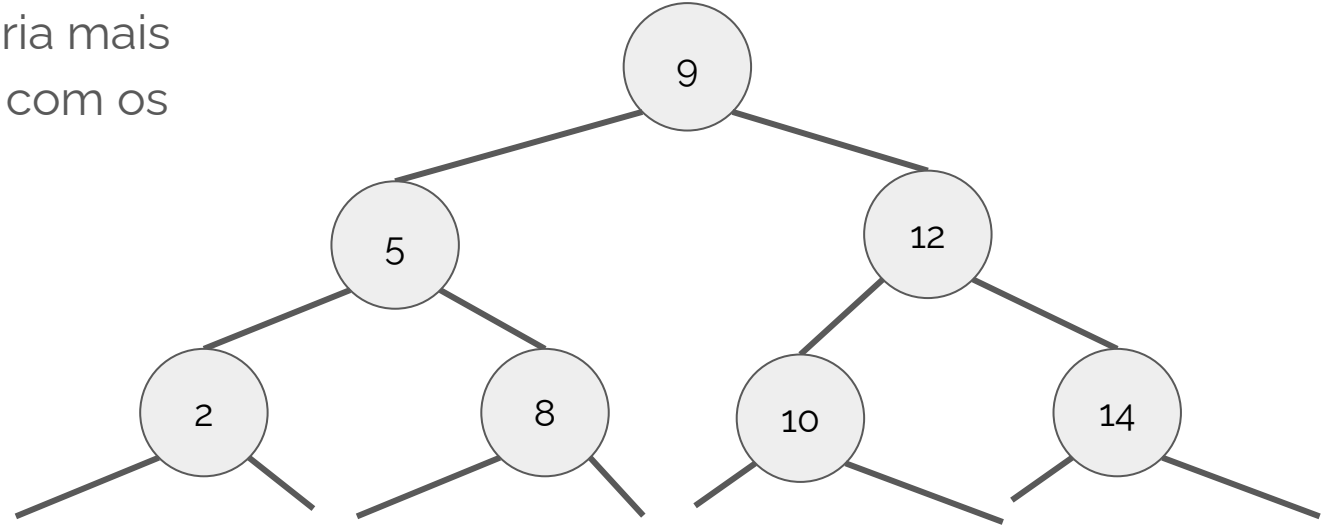
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

árvores binárias balanceadas

Árvores Binárias: Construção

Crie uma árvore binária mais balanceada possível com os elementos a seguir

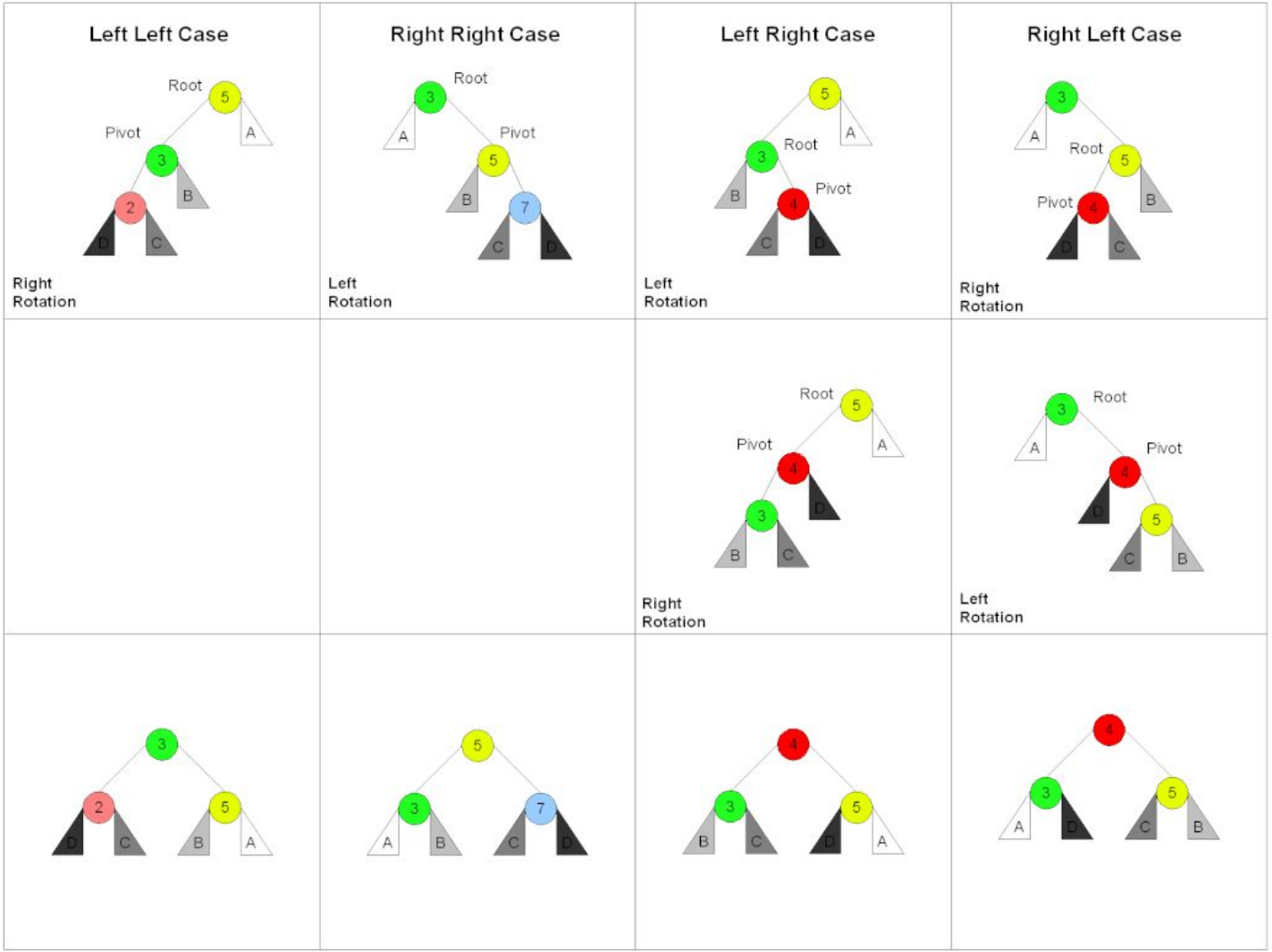
2
5
8
9
10
12
14



Árvores Binárias: Balanceamento

É possível reconfigurar uma árvore para balancear sua distribuição

Árvores Binárias:
Balanceamento



. . .

armazenamento e compressão

Armazenamento e Compressão : ASCII

127 caracteres

quantos bits por char?

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Armazenamento e Compressão : ASCII

Pra 7 caracteres
3 bits por caractere
50.000 caracteres
150.000 bits

mas se desses 7 só 3 fossem usados com frequência, e os outros 4 fossem usados raramente, daria para economizar espaço?

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Como um exemplo trivial, suponha que uma coleção de dados contém 50.000 vezes os seis caracteres *a*, *c*, *g*, *k*, *p* e ?, que ocorrem com as seguintes frequências percentuais:

Caractere	*a*	*c*	*g*	*k*	*p*	?
Frequência	48	9	12	4	17	10

Como seis caracteres distintos têm que ser armazenados, o esquema de comprimento fixo necessitaria de um mínimo de três bits por caractere ($2^3 = 8 \geq 6$). O espaço total necessário seria então de $50.000 * 3 = 150.000$ bits. Suponha agora que, EM VEZ desse esquema, usássemos o seguinte esquema de codificação:

Caractere	*a*	*c*	*g*	*k*	*p*	?
Esquema de codificação	0	1101	101	1100	111	100

O espaço necessário (número de bits) seria, então,

$$50.000\ (0{,}48 * 1 + 0{,}09 * 4 + 0{,}12 * 3 + 0{,}04 * 4 + 0{,}17 * 3 + 0{,}10 * 3) = 108.500$$

que é, aproximadamente, dois terços do espaço anterior.

Como um exemplo trivial, suponha que uma coleção de dados contém 50.000 vezes os seis caracteres *a*, *c*, *g*, *k*, *p* e ?, que ocorrem com as seguintes frequências percentuais:

Caractere	*a*	*c*	*g*	*k*	*p*	?
Frequência	48	9	12	4	17	10

Como seis caracteres distintos têm que ser armazenados, o esquema de comprimento fixo necessitaria de um mínimo de três bits por caractere ($2^3 = 8 \geq 6$). O espaço total necessário seria então de $50.000 * 3 = 150.000$ bits. Suponha agora que, EM VEZ desse esquema, usássemos o seguinte esquema de codificação:

Caractere	*a*	*c*	*g*	*k*	*p*	?
Esquema de codificação	0	1101	101	1100	111	100

O espaço necessário (número de bits) seria, então,

$$50.000\ (0{,}48 * 1 + 0{,}09 * 4 + 0{,}12 * 3 + 0{,}04 * 4 + 0{,}17 * 3 + 0{,}10 * 3) = 108.500$$

que é, aproximadamente, dois terços do espaço anterior.

Que código é esse?

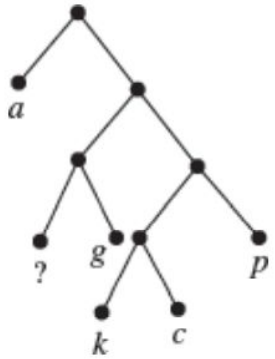
a. 11111111010100
b. 1101010101100
c. 100110001101100

código de huffman

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

codificação com árvore binária

Em nosso estudo de códigos de prefixo, construiremos árvores binárias tendo os caracteres como folhas. Uma vez construída a árvore, um código binário pode ser atribuído a cada caractere simplesmente percorrendo-se o caminho da raiz até a folha correspondente, usando 0 para uma ramificação da esquerda e 1 para uma da direita. Como nenhuma folha precede outra em algum caminho começando na raiz, o código será um código de prefixo. A Figura 6.57 ilustra a árvore binária para o código do Exemplo 37.



Algoritmo de Codificação de Huffman

Suponha então que temos m caracteres em um arquivo e que conhecemos a frequência percentual de cada um deles. O algoritmo para se construir a árvore funciona mantendo-se uma lista L de nós que são raízes de árvores binárias. Inicialmente, L contém m raízes, cada uma rotulada com a frequência de um dos caracteres; as raízes serão ordenadas pela frequência em ordem crescente, e nenhuma delas terá filho. Segue uma descrição do algoritmo em pseudocódigo.

```
ÁrvoreDeHuffman (lista de nós L; inteiro m)
// A cada um dos m nós em L está associada uma frequência f, e L é ordenada
// pela frequência em ordem crescente; o algoritmo constrói a árvore de Huffman

  para i = 1 até m − 1 faça
    crie novo nó z
    sejam x, y os dois primeiros nós em L    //nós de frequência mínima
    f (z) = f (x) + f (y)
    insira z em ordem em L
    filho esquerdo de z = nó x
    filho direito de z = nó y //x, y não pertencem mais a L
  fim do para
fim da ÁrvoreDeHuffman
```

Algoritmo de Codificação de Huffman

Suponha então que temos m caracteres em um arquivo e que conhecemos a frequência percentual de cada um deles. O algoritmo para se construir a árvore funciona mantendo-se uma lista L de nós que são raízes de árvores binárias. Inicialmente, L contém m raízes, cada uma rotulada com a frequência de um dos caracteres; as raízes serão ordenadas pela frequência em ordem crescente, e nenhuma delas terá filho. Segue uma descrição do algoritmo em pseudocódigo.

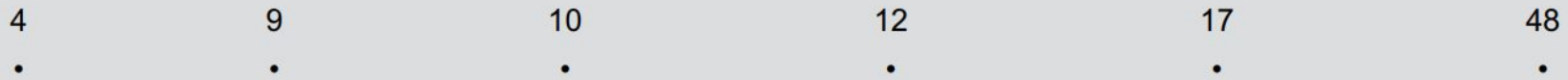
```
ÁrvoreDeHuffman (lista de nós L; inteiro m)
// A cada um dos m nós em L está associada uma frequência f, e L é ordenada
// pela frequência em ordem crescente; o algoritmo constrói a árvore de Huffman

  para i = 1 até m − 1 faça
    crie novo nó z
    sejam x, y os dois primeiros nós em L    //nós de frequência mínima
    f (z) = f (x) + f (y)
    insira z em ordem em L
    filho esquerdo de z = nó x
    filho direito de z = nó y //x, y não pertencem mais a L
  fim do para
fim da ÁrvoreDeHuffman
```

executando o algoritmo para os caracteres anteriores nas frequências apontadas

a	*c*	*g*
48	9	12

k	*p*	?
4	17	10

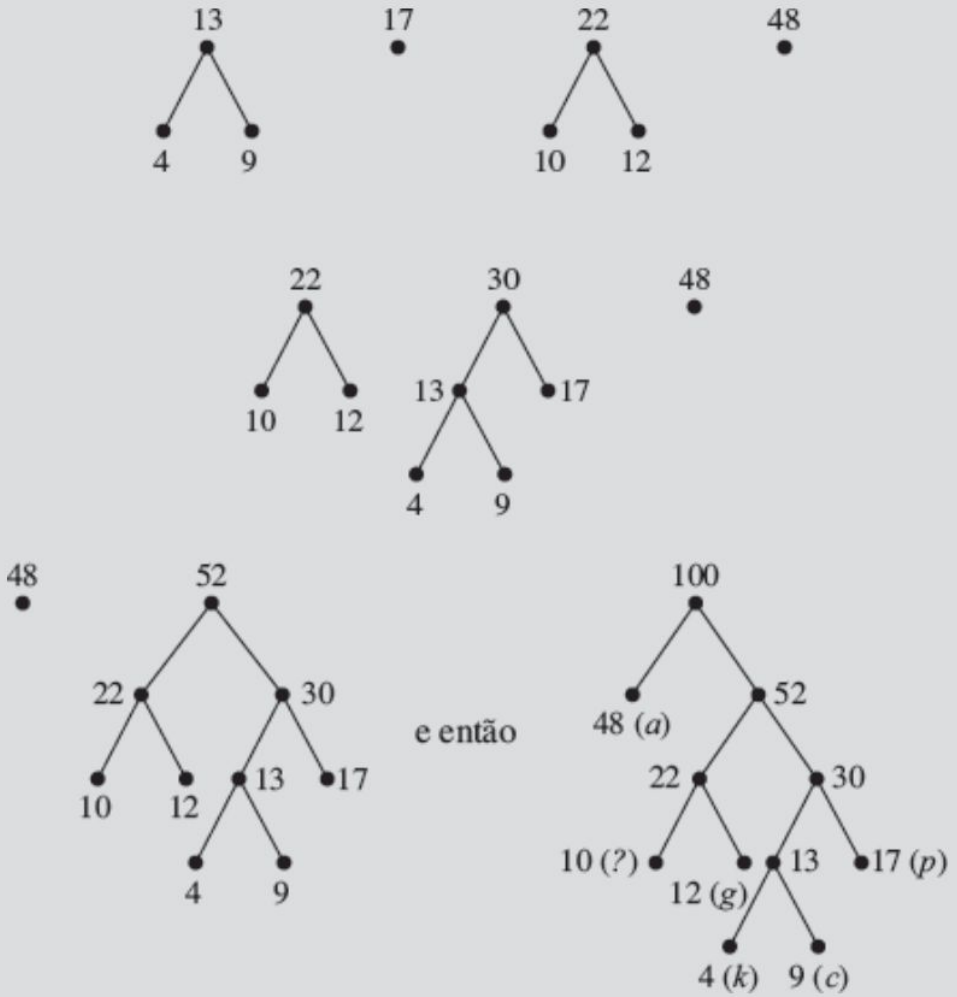
Vamos usar o algoritmo *ÁrvoreDeHuffman* para construir a árvore na Figura 6.57, que é baseada nos dados do Exemplo 37. Inicialmente, *L* contém seis nós, ordenados por frequência:



Seguindo o algoritmo, entramos no laço do **para** pela primeira vez. Os nós x e y são os com frequência 4 e 9, respectivamente. Um novo nó com frequência $4 + 9 = 13$ é criado e inserido, em ordem, na lista L, com o nó x como seu filho esquerdo e o nó y como seu filho direito. A nova lista L fica assim:

Esse processo é repetido mais quatro vezes. As listas novas L resultantes em cada etapa são:

Esse processo é repetido mais quatro vezes. As listas novas *L* resultantes em cada etapa são:



Etapas da Codificação

Codificação Etapa 1	No arquivo original TEXTOCOMUM, faça uma análise de frequência, ou seja, crie um arquivo FREQUÊNCIA que contém dados da forma *a*—18 *b*—7 e assim por diante.
Codificação Etapa 2	Usando FREQUÊNCIA, crie um arquivo TABELADECÓDIGOS que contém o código de Huffman para cada caractere, por exemplo, *a*—001 *b*—1110 e assim por diante.
Codificação Etapa 3	Usando TEXTOCOMUM e TABELADECÓDIGOS, crie um arquivo chamado CODIFICADO que contém os dados comprimidos.
Decodificação	Usando CODIFICADO e TABELADECÓDIGOS, decodifique o texto para recuperar TEXTOCOMUM.

O arquivo CODIFICADO é a versão com compressão de dados de TEXTOCOMUM e, presume-se, necessita de menos espaço. No entanto, o arquivo TABELADECODIGOS também precisa ser armazenado para se poder decodificar o arquivo.

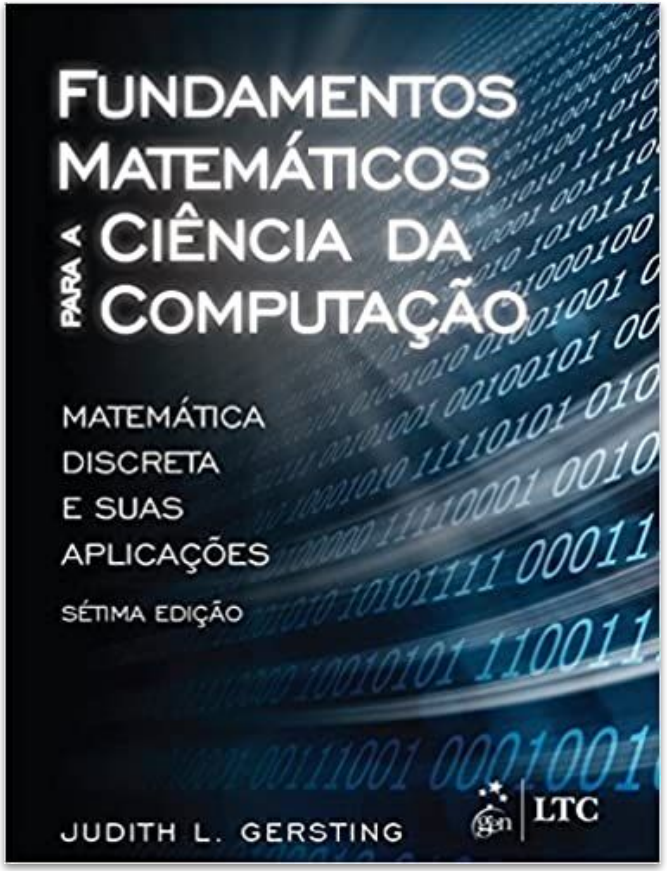
Livros > *Slides*

Universities should ban PowerPoint
https://www.businessinsider.com/universities-should-ban-powerpoint-it-makes-students-stupid-and-professors-boring-2015-6

Craig, Russell J., and Joel H. Amernic. "PowerPoint presentation technology and the dynamics of teaching." Innovative Higher Education 31.3 (2006): 147-160.

Referência

Fundamentos Matemáticos para Ciência da Computação (3a Edição)

5. Grafos e Árvores

Seção 5.4: Terminologia e Aplicações de Grafos